Day 107: Random Forests From Scratch — Bagging and a Real Variance-Reduction Demo

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 16, Day 3 | Complete study guide

Watch: StatQuest: Random Forests Part 1 - Building, Using and Evaluating | 22. Bagging and Random Forests (Inside Bloomberg)

Script: `day107_random_forests.py` · every number in this guide is copied by program from `day107_run_output.txt`, the real output of that script.

A random forest written from scratch on Day 105's split search. It matches scikit-learn's forest on breast cancer (0.9608 against 0.9614). Averaging is measured directly: on repeated training sets the variance of a prediction falls from 0.1258 for one tree to 0.0224 for 64, while the bias stays where it was. The honest limit: on noisy Titanic a default forest (0.7996) does not beat one pruned tree (0.8093) unless its leaves are limited (0.8209). On digits the forest wins by +0.1224.

0. Where today fits

DEFINITION — start here, no prior knowledge assumed

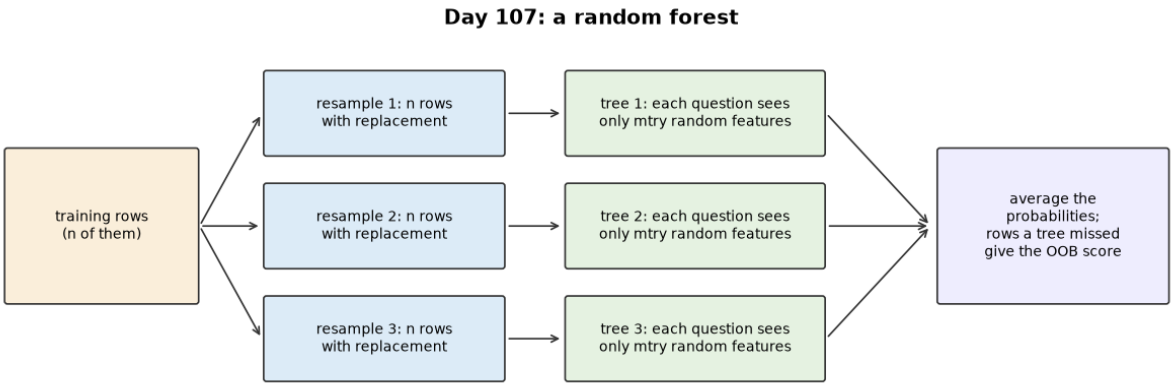
Bagging (bootstrap aggregating) trains many models, each on a resample of the rows, and averages them.

A **random forest** is bagged trees with one more idea: each question may consider only **mtry** random features, so the trees differ more. **Variance** here means how much a prediction changes when the training data changes.

REAL-WORLD SCENARIO

Asking a crowd to guess the number of sweets in a jar. Each guess is poor in its own way, but the errors point in different directions, so the average is usually better than most single guesses.

SYSTEM DIAGRAM



A random forest: resample the rows for each tree, limit the features at each question, average the answers.

Today's part for Day 111

`day107_exercise.py` is a small complete forest, using Day 105's exercise for the split search. Its real output:

ACTUAL OUTPUT

```
eight patients, 25 trees, mtry 1: training accuracy 1.000, OOB accuracy 1.000
probabilities sum to 1: largest error 0.0e+00
same seed: largest difference 0.0e+00; another seed: 0.200
rows left out per tree (expected about 8 x 0.34 = 2.7): 3.1
mtry 1 with a constant column: the root still uses the real feature in 20 of 20 seeds
```

Data: Titanic (the repository's own copy), and the breast-cancer and digits sets bundled with scikit-learn. The forest is written in NumPy; scikit-learn's forest is run beside it only as an outside check.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. The bootstrap

DEFINITION — start here, no prior knowledge assumed

A **bootstrap resample** draws n rows from n rows *with replacement*, so some rows appear twice and others not at all. One row is missed by all n draws with probability $(1 - 1/n)^n$.

REAL-WORLD SCENARIO

Pulling a name from a hat n times, returning it after each draw: some people are called twice and others never.

MATHEMATICS

$$P(\text{a row is left out}) = (1 - 1/n)^n \to 1/e \approx 0.368 \text{ as } n \text{ grows}$$

ACTUAL OUTPUT

```
    rows  distinct share   (1 - 1/n)^n left out
      10          0.6459                 0.3487
     100          0.6341                 0.3660
     398          0.6327                 0.3674
   10000          0.6323                 0.3679
as n grows the share left out approaches 1/e = 0.3679, so each tree never sees about a third of
    the rows
Actual output from this lesson's run.
```

What it means

The measured share of distinct rows is about 0.6327 at 398 rows, and the left-out share is close to 0.3679 even for small tables. Every tree therefore has about a third of the rows it never saw, which section 4 turns into a free test set.

WHY CHOOSE THIS?

Each tree gets slightly different data, cheaply, with nothing thrown away overall.

WHY NOT THIS? (TRADEOFFS)

Each tree sees fewer distinct rows, so a single bagged tree is a little weaker.

2. Correctness checks

DEFINITION — start here, no prior knowledge assumed

With every feature allowed, no resampling and features in the usual order, the forest's tree must be Day 105's tree. scikit-learn's forest with the same settings gives an outside check.

REAL-WORLD SCENARIO

Checking a new scale against a known weight, and then against a second scale.

IMPLEMENTATION

```
def grow_random(X, y, n_classes, mtry, rng, depth=0, max_depth=None, min_leaf=1):
    counts = np.bincount(y, minlength=n_classes)
    node = {"counts": counts, "pred": int(np.argmax(counts))}
    if (max_depth is not None and depth >= max_depth) or counts.max() == len(y):
        return node
    order = rng.permutation(X.shape[1])
    feats = order[:mtry]; s = best_split(X[:, feats], y, n_classes, gini, min_leaf); k = mtry
    while s is None and k < X.shape[1]:
        feats = order[k:k + 1]; s = best_split(X[:, feats], y, n_classes, gini, min_leaf); k += 1
    if s is None:
        return node
    f, t = int(feats[s[0]]), s[1]; m = X[:, f] <= t
    node.update(feature=f, threshold=t, gain=s[2],
                left=grow_random(X[m], y[m], n_classes, mtry, rng, depth + 1, max_depth,
                    min_leaf),
                right=grow_random(X[~m], y[~m], n_classes, mtry, rng, depth + 1, max_depth,
                    min_leaf))
    return node
```

ACTUAL OUTPUT

```
mtry = all features, no bootstrap, features in Day 105's order: 1.0000 of test predictions equal Day 105's
    tree
the same with features in a random order: 0.9591, because tied splits go to whichever feature is tried first
forest probabilities sum to 1 for every row: largest error 0.0e+00
breast cancer, 10 splits, 100 trees: from scratch 0.9608, scikit-learn 0.9614, mean gap -0.0006
Actual output from this lesson's run.
```

What it means

In Day 105's order the predictions match exactly (1.0000). In a random order they do not (0.9591), because tied splits go to whichever feature is tried first, which is a small lesson in itself: even one 'deterministic' tree depends on arbitrary choices. Against scikit-learn the mean gap over 10 splits is -0.0006.

WHY CHOOSE THIS?

One change at a time: the tree is verified before randomness is added.

WHY NOT THIS? (TRADEOFFS)

Matching an average does not prove the two forests match tree by tree.

3. Variance, measured

DEFINITION — start here, no prior knowledge assumed

Thirty different training sets each get a forest, and all predict the same test passengers. For one passenger, the spread of those predictions is the **variance**. The squared distance between their average and the true label holds the **bias** and the unavoidable noise in the label. The two add up to the **Brier score**, the squared error of the probability.

REAL-WORLD SCENARIO

Thirty valuers price the same house. How much they differ from each other is one problem; whether their average is wrong is another.

MATHEMATICS

$$Brier = variance + bias^2 + noise$$

$$Var(\text{average of } B \text{ trees}) = \rho\sigma^2 + (1 - \rho)\,\sigma^2 / B$$

σ^2 = *one tree's variance;* ρ = *the correlation between two trees of a forest*

SPECIFICATION

Training sets	30 random subsets of 300 passengers from Day 105's 623 training rows.
Test rows	The same 268 passengers every time.
Note	The 30 subsets overlap, so true variance across new datasets would be higher.

IMPLEMENTATION

```
q = P - P.mean((0, 1), keepdims=True)
sig = float((q ** 2).mean())
s = q.sum(1)
cov = float(((s ** 2 - (q ** 2).sum(1)) / (BAG * (BAG - 1))).mean())
rho = cov / sig
```

ACTUAL OUTPUT

```
Titanic: 30 random training sets of 300 passengers from the 623 training rows, the same 268 test passengers,
    64 trees each
per test row and across training sets: variance of p(survived), and (mean p - label)^2, which holds the bias
    and the noise in the label
bagging (all 6 features)
  trees   variance  bias+noise    Brier  accuracy
      1     0.1258      0.1452   0.2710    0.7274
      2     0.0736      0.1448   0.2184    0.7541
      4     0.0463      0.1399   0.1861    0.7678
      8     0.0339      0.1398   0.1737    0.7739
     16     0.0270      0.1393   0.1664    0.7705
     32     0.0238      0.1400   0.1639    0.7716
     64     0.0224      0.1405   0.1628    0.7721
one tree's variance 0.1247; correlation between two trees of a forest 0.166; so the floor for many trees is
    rho * sigma^2 = 0.0207
```

ACTUAL OUTPUT

```
forest (mtry 2)
  trees   variance  bias+noise    Brier  accuracy
      1     0.1311      0.1423   0.2734    0.7255
      2     0.0716      0.1405   0.2121    0.7475
      4     0.0454      0.1375   0.1829    0.7644
      8     0.0304      0.1369   0.1673    0.7780
     16     0.0236      0.1361   0.1596    0.7787
     32     0.0197      0.1360   0.1558    0.7830
     64     0.0181      0.1351   0.1532    0.7868
one tree's variance 0.1327; correlation between two trees of a forest 0.123; so the floor for many trees is
    rho * sigma^2 = 0.0163
Actual output from this lesson's run.
```

What it means

This is the whole idea in one table. From 1 tree to 64 the variance falls from 0.1258 to 0.0224, while bias+noise barely moves (0.1452 to 0.1405): averaging cures variance and does nothing for bias. Accuracy improves as a result, from 0.7274 to 0.7721. It does not fall forever: with correlation 0.166, the floor is 0.0207. Limiting the features lowers the correlation to 0.123 and the floor to 0.0163. That formula holds by construction, so the point is not that it fits, but what it says: only less correlated trees can go lower.

WHY CHOOSE THIS?

Measures the claim directly instead of assuming it.

WHY NOT THIS? (TRADEOFFS)

Thirty overlapping subsets of one table: the size of the effect is specific to this setup.

4. How many trees?

DEFINITION — start here, no prior knowledge assumed

A forest grows one tree at a time, so accuracy can be read at every size. The **out-of-bag** (OOB) score judges each row using only the trees that never saw it.

REAL-WORLD SCENARIO

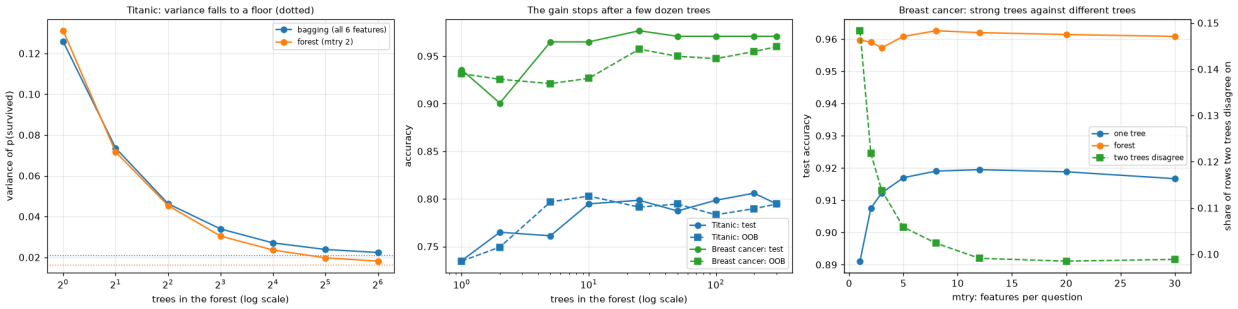
Polling more people: the first hundred change the estimate a lot, the ten-thousandth hardly at all.

IMPLEMENTATION

```
def oob_accuracy(trees, bags, X, y, n_classes):
    votes = np.zeros((len(y), n_classes))
    for t, idx in zip(trees, bags):
        out = np.setdiff1d(np.arange(len(y)), idx)
        votes[out] += proba(t, X[out], n_classes)
    seen = votes.sum(1) > 0
    return float(np.mean(np.argmax(votes[seen], 1) == y[seen])), float(seen.mean())
```

ACTUAL OUTPUT

```
Titanic: 623 training, 268 test
  trees  test acc   OOB acc  rows with an OOB vote
      1    0.7351    0.7346                 0.3387
      2    0.7649    0.7493                 0.5827
      5    0.7612    0.7968                 0.9005
     10    0.7948    0.8029                 0.9856
     25    0.7985    0.7913                 1.0000
     50    0.7873    0.7945                 1.0000
    100    0.7985    0.7833                 1.0000
    200    0.8060    0.7897                 1.0000
    300    0.7948    0.7945                 1.0000
Breast cancer: 398 training, 171 test
  trees  test acc   OOB acc  rows with an OOB vote
      1    0.9357    0.9315                 0.3668
      2    0.9006    0.9256                 0.6080
      5    0.9649    0.9211                 0.8920
     10    0.9649    0.9266                 0.9925
     25    0.9766    0.9573                 1.0000
     50    0.9708    0.9497                 1.0000
    100    0.9708    0.9472                 1.0000
    200    0.9708    0.9548                 1.0000
    300    0.9708    0.9598                 1.0000
Actual output from this lesson's run.
```



Left: variance against forest size, with the floors dotted. Middle: test and OOB accuracy. Right: mtry on breast cancer.

What it means

Most of the gain comes in the first few dozen trees; after that the curves only wander. Small forests can even dip: breast cancer falls from 0.9357 with one tree to 0.9006 with two. After that, the widely quoted claim that more trees do not overfit holds here, in the sense that nothing gets steadily worse. Extra trees cost time and do little else.

WHY CHOOSE THIS?

OOB needs no held-out data, and arrives for free with the training.

WHY NOT THIS? (TRADEOFFS)

It is noisy on small data, and it needs enough trees that every row has a vote.

5. OOB as a free validation set: does a forest still overfit?

DEFINITION — start here, no prior knowledge assumed

A forest cannot overfit by having too many *trees*, but each tree can still memorise rows. A **minimum leaf size** limits that, and OOB can choose it without cross-validation.

REAL-WORLD SCENARIO

A panel of judges can still all be misled if each one trusts a single anecdote.

SPECIFICATION

Data	Titanic, 10 splits, 100 trees, mtry 2.
Choice	On each split the leaf size with the best OOB accuracy; test used once.

ACTUAL OUTPUT

```
Titanic, 10 splits, 100 trees, mtry 2: forests with a minimum leaf size, unpruned at 1
 min leaf  train acc  OOB acc  test acc  OOB - test
        1     0.9844   0.8082    0.8045     +0.0037
        2     0.9432   0.8207    0.8183     +0.0024
        5     0.8953   0.8260    0.8209     +0.0051
       10     0.8709   0.8157    0.8160     -0.0003
       20     0.8417   0.8066    0.8078     -0.0013
       40     0.8116   0.7872    0.7799     +0.0073
choosing the leaf size by OOB on each split: test 0.8175, against 0.8045 at leaf size 1; no
    cross-validation needed
sizes chosen: [5, 2, 5, 5, 5, 5, 10, 1, 5, 10]
Actual output from this lesson's run.
```

What it means

Training accuracy of 0.9844 against a test score of 0.8045 shows that an unpruned forest is memorising. Limiting leaves to 5 reaches 0.8209. OOB tracks test closely, within +0.0073 at every size, so it chooses well: 0.8175 with no cross-validation at all. That answers the awkward result of section 8: on noisy data the default needs this setting.

WHY CHOOSE THIS?

Tuning from one fit: no folds, no extra training for validation.

WHY NOT THIS? (TRADEOFFS)

The default of fully grown trees is not always right, though it is often quoted as if it were.

6. mtry: strong trees against different trees

DEFINITION — start here, no prior knowledge assumed

Allowing fewer features per question makes each tree weaker but the trees more different. A forest wants both strong and different trees, so mtry trades one for the other.

REAL-WORLD SCENARIO

A committee of specialists who each read only part of the file: each knows less, and they share fewer blind spots.

ACTUAL OUTPUT

```
breast cancer, 10 splits, 100 trees: 'disagree' is the share of test rows on which two trees of a
    forest differ
  mtry  one tree  disagree   forest
     1    0.8911    0.1483   0.9596
     2    0.9075    0.1218   0.9591
     3    0.9123    0.1138   0.9573
     5    0.9170    0.1058   0.9608
     8    0.9190    0.1024   0.9626
    12    0.9195    0.0991   0.9620
    20    0.9188    0.0985   0.9614
    30    0.9167    0.0989   0.9608
mtry 5 is the usual default for 30 features; mtry 30 is plain bagging
Actual output from this lesson's run.
```

What it means

The trade-off is visible: going from mtry 30 to 1, one tree weakens from 0.9167 to 0.8911 and disagreement rises from 0.0989 to 0.1483. But the forest barely moves, between 0.9573 and 0.9626. On this data mtry hardly matters. Section 8 shows where it does.

WHY CHOOSE THIS?

Often the only forest setting worth tuning.

WHY NOT THIS? (TRADEOFFS)

On many tables the default is fine, and tuning it fits noise.

7. Useless features: where the random subsets hurt

DEFINITION — start here, no prior knowledge assumed

If most columns are noise, a random subset may contain few useful ones, and a question has to split on something worse.

REAL-WORLD SCENARIO

Drawing six advisers at random from a room in which most know nothing.

ACTUAL OUTPUT

```
breast cancer with random columns added, 5 splits, 100 trees; a question may see sqrt(p) features, or all of
    them
  added     p  sqrt(p)  chance of a real one  one tree   forest  bagging
     0    30        5                 1.000    0.9322   0.9591   0.9661
    10    40        6                 1.000    0.9287   0.9602   0.9626
    30    60        7                 0.995    0.9287   0.9544   0.9579
   100   130       11                 0.951    0.9287   0.9497   0.9591
   300   330       18                 0.829    0.9240   0.9497   0.9567
Actual output from this lesson's run.
```

What it means

With 300 useless columns, a question has only a 0.829 chance of seeing even one real feature. The forest falls from 0.9591 to 0.9497, while bagging, which sees every feature, holds 0.9567. These are 5 splits, so read the direction, not the third decimal. Even so, the forest is still clearly above one tree. Random subsets are helpful, not free.

WHY CHOOSE THIS?

Robust: hundreds of useless columns cost about a point here.

WHY NOT THIS? (TRADEOFFS)

When signal is rare, plain bagging or a larger mtry can do better.

8. Is it better?

DEFINITION — start here, no prior knowledge assumed

The same splits for every method: Day 105's full tree, Day 106's pruned tree, bagging, a random forest, and scikit-learn's forest.

REAL-WORLD SCENARIO

A league table, on three different pitches.

SPECIFICATION

Splits	Titanic and breast cancer 20, digits 6 (it is slower).
Forests	100 trees, default fully grown trees.
Intervals	Day 99's bootstrap over the splits. The splits share rows, so they are too narrow.

ACTUAL OUTPUT

```
Titanic: 891 rows, 6 features, 2 classes, 20 splits, mtry 2
  full tree             0.7662  (std 0.0182)
  pruned tree           0.8093  (std 0.0253)
  bagging               0.7996  (std 0.0217)
  random forest         0.7996  (std 0.0208)
  scikit-learn forest   0.7998  (std 0.0198)
  forest minus pruned tree  -0.0097 [-0.0201, +0.0002]; better on 9, worse on 9 of 20
  forest minus bagging      +0.0000 [-0.0050, +0.0050]; better on 9, worse on 8 of 20
Breast cancer: 569 rows, 30 features, 2 classes, 20 splits, mtry 5
  full tree             0.9193  (std 0.0166)
  pruned tree           0.9243  (std 0.0147)
  bagging               0.9553  (std 0.0146)
  random forest         0.9596  (std 0.0128)
  scikit-learn forest   0.9599  (std 0.0134)
  forest minus pruned tree  +0.0354 [+0.0284, +0.0418]; better on 20, worse on 0 of 20
  forest minus bagging      +0.0044 [-0.0009, +0.0096]; better on 11, worse on 6 of 20
Digits: 1797 rows, 64 features, 10 classes, 6 splits, mtry 8
  full tree             0.8503  (std 0.0129)
  pruned tree           0.8488  (std 0.0201)
  bagging               0.9511  (std 0.0091)
  random forest         0.9712  (std 0.0054)
  scikit-learn forest   0.9753  (std 0.0023)
  forest minus pruned tree  +0.1224 [+0.1061, +0.1367]; better on 6, worse on 0 of 6
  forest minus bagging      +0.0201 [+0.0130, +0.0269]; better on 6, worse on 0 of 6
the splits share rows, so these intervals are too narrow, a rough guide only
Actual output from this lesson's run.
```

What it means

Three datasets, three answers. **Digits:** a forest transforms the result, 0.8488 to 0.9712, and here mtry matters (+0.0201 over bagging, on all 6 splits). **Breast cancer:** a clear win of +0.0354. **Titanic:** no win. The pruned tree scores 0.8093 and the forest 0.7996, because with 6 features and noisy labels there is little to gain from diversity, and fully grown trees memorise; section 5's leaf limit recovers it. Our digits forest (0.9712) sits a little below scikit-learn's (0.9753) on 6 splits; I did not investigate why.

WHY CHOOSE THIS?

Very good results with almost no tuning, on all kinds of tables.

WHY NOT THIS? (TRADEOFFS)

Slow to predict and impossible to read as rules; Day 108 asks what such a model is using.

Interview preparation — technical questions

Q: What share of rows does a bootstrap contain?

A: About 63.2%; the other 36.8% are out-of-bag for that tree.

Q: What is OOB error?

A: Each row is scored only by trees that never saw it: a validation score with no held-out set.

Q: What is mtry, and what is the default?

A: Features considered per question: usually the square root of p for classification.

Q: Why average probabilities rather than votes?

A: It keeps how sure each tree was. With fully grown trees the two are close.

Q: How do you limit overfitting in a forest?

A: A minimum leaf size or a depth limit, not fewer trees. Here leaf size 5 lifted Titanic from 0.8045 to 0.8209.

Interview preparation — theoretical questions

Q: Why does averaging reduce variance but not bias?

A: Averaging keeps the mean prediction and shrinks the spread: here 0.1258 to 0.0224, with bias+noise unchanged.

Q: Why not just add trees forever?

A: Variance stops at the floor rho times sigma squared; only less correlated trees go lower.

Q: Bagging or a random forest?

A: A forest when strong features would otherwise dominate every tree (digits, +0.0201). Bagging can be better with many useless columns.

Q: Does a random forest always beat one tree?

A: No. On Titanic a pruned tree scored 0.8093 against a default forest's 0.7996.

Key takeaway and what's next

> **KEY TAKEAWAY**
>
> A forest fights variance, not bias. Measured on repeated training sets, averaging cut the variance of a prediction by more than five times and left bias where it was. That helps most where trees are unstable and the data is rich, and little on a small, noisy table whose signal one pruned tree can already reach. The defaults are good, but not sacred.

- A bootstrap leaves out about 0.368 of the rows; from scratch matches scikit-learn on breast cancer (gap -0.0006).
- Titanic: variance 0.1258 to 0.0224 from 1 to 64 trees; bias+noise about 0.14 throughout.
- OOB within 0.0073 of test; choosing the leaf size by OOB gave 0.8175 against 0.8045.
- Forest against pruned tree: digits +0.1224, breast cancer +0.0354, Titanic -0.0097.

Suggested watch order, if starting from zero

- 1. StatQuest: Random Forests Part 1 - Building, Using and Evaluating — how a forest is built and evaluated.
- 2. 22. Bagging and Random Forests (Inside Bloomberg) — a lecture on bagging and why it reduces variance.

Day 108 preview

- **Feature importance.** A forest cannot be read like one tree, so which features did it use? Impurity-based importance against permutation importance, on cases where the right answer is known.